JVM
パフォーマンスチューニング
基礎

2013/12/14

せとあずさ♂

- @setoazusa
- http://blog.fieldnotes.jp/
- #tddbc 横浜(2011～2013)
- #java_ja
- #yokohamarb
- とはいえ、メインはJava
- 最近、MacからWindows8に乗り換えました
- チャンキヨかわいいよチャンキヨ
- miwaは自慢の妹です

おことわり

- 今日は「JVMパフォーマンスチューニング基礎」というお題ですが…
  - →HotSpot VMに限定した話になります m(__)m

アジェンダ

- パフォーマンスチューニングの原則
- ツールあれこれ(小ネタ)
- サーバーVMとクライアントVM
- エルゴノミクス

パフォーマンスチューニングの原則

- 測定する

測定に使用するツール

- サーバー側
  - VisualVM
  - jstat
  - sysstat / vmstat
  - SQLのログ(呼び出し回数、スロークエリー)
- クライアント側
  - Jmeter
  - ab
  - gatling

(小ネタ)visualvmでリモートサーバーのヒープの状態を監視する。

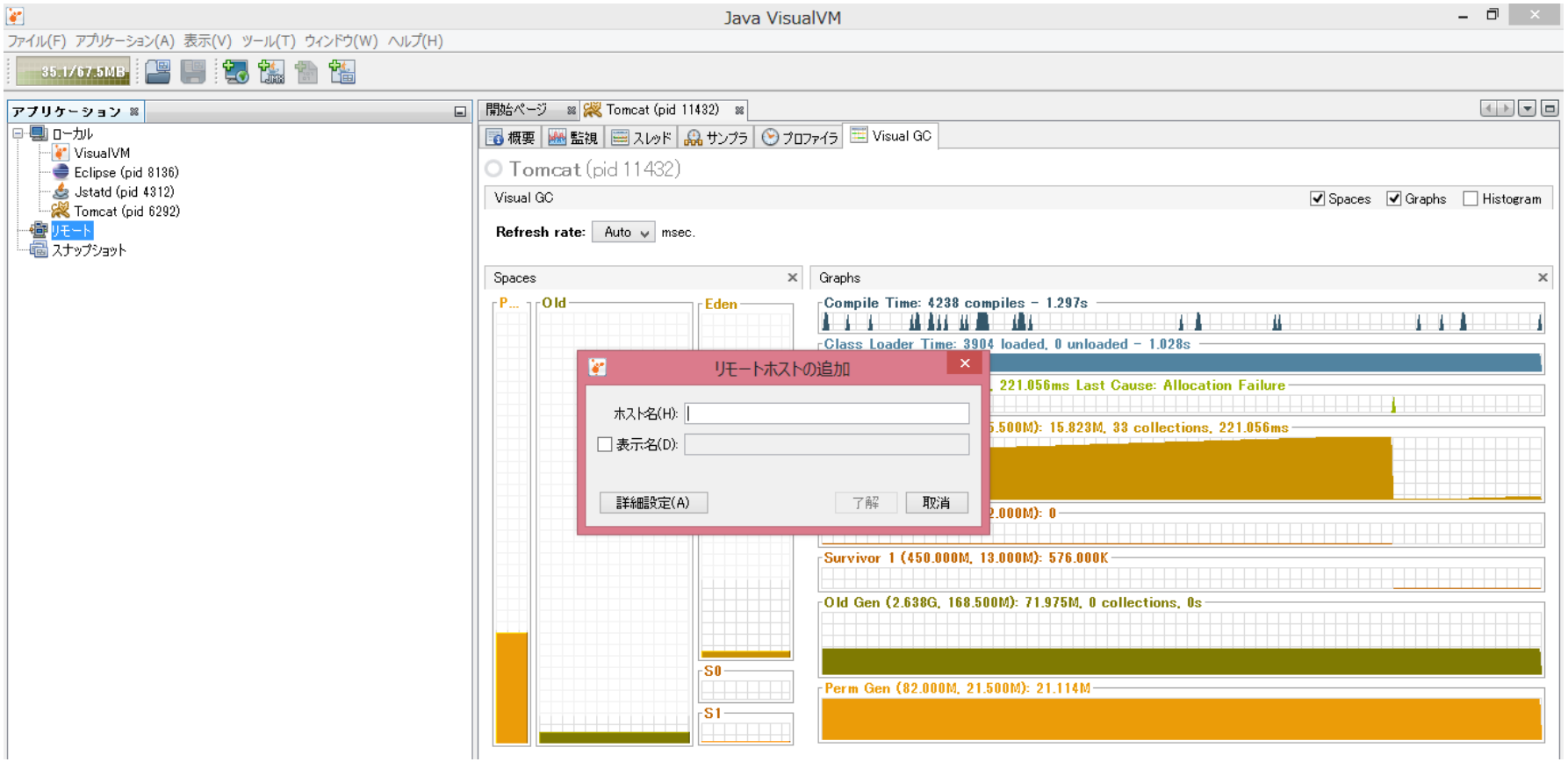
- こんな感じのpolicyファイルを書いて、

```
grant codebase "file:${java.home}/../lib/tools.jar" {
   permission java.security.AllPermission;
};
```

- jstatdを起動します。

```
C:¥usr>jstatd.exe -J-Djava.security.policy=jstatd.all.policy -J-Djava.rmi.server.hostname=192.168.0.4
```

後はVisualVMでリモートホストを追加すればOK。



(小ネタ)jstatの出力にタイムスアンプをつける

```
p=`ps aux |grep tomcat |grep うにゃうにゃ|grep java |awk '{print $2}'`; jstat -gcutil -h10 $p 10000 | awk '{print strftime("%H:%M:%S"), $0}' >>jstat.log
```

クライアントVMとサーバーVM

- クライアントVM
  - 起動時間を短縮し、メモリサイズを縮小するように調整されています。
- サーバーVM
  - プログラム実行速度が最大になるように設計されています。

- 64bitのHotSportVMには、サーバーVMしか呼び出しオプションがありません。(-clientを指定してもサーバVMが起動します)
- じゃあ、サーバーVMしか実装がいないのかというと、そうではなくて...

- サーバーVMで、起動時のオーバーヘッドを短縮する方法→階層型コンパイル
  - JDK7u1 JDK6u25 から導入
  - 最初はクライアントVMでコンパイルして、呼び出し回数に応じてサーバーVMに切り替える
  - -XX:+TieredCompilation -XX:TieredStopAtLevel=1

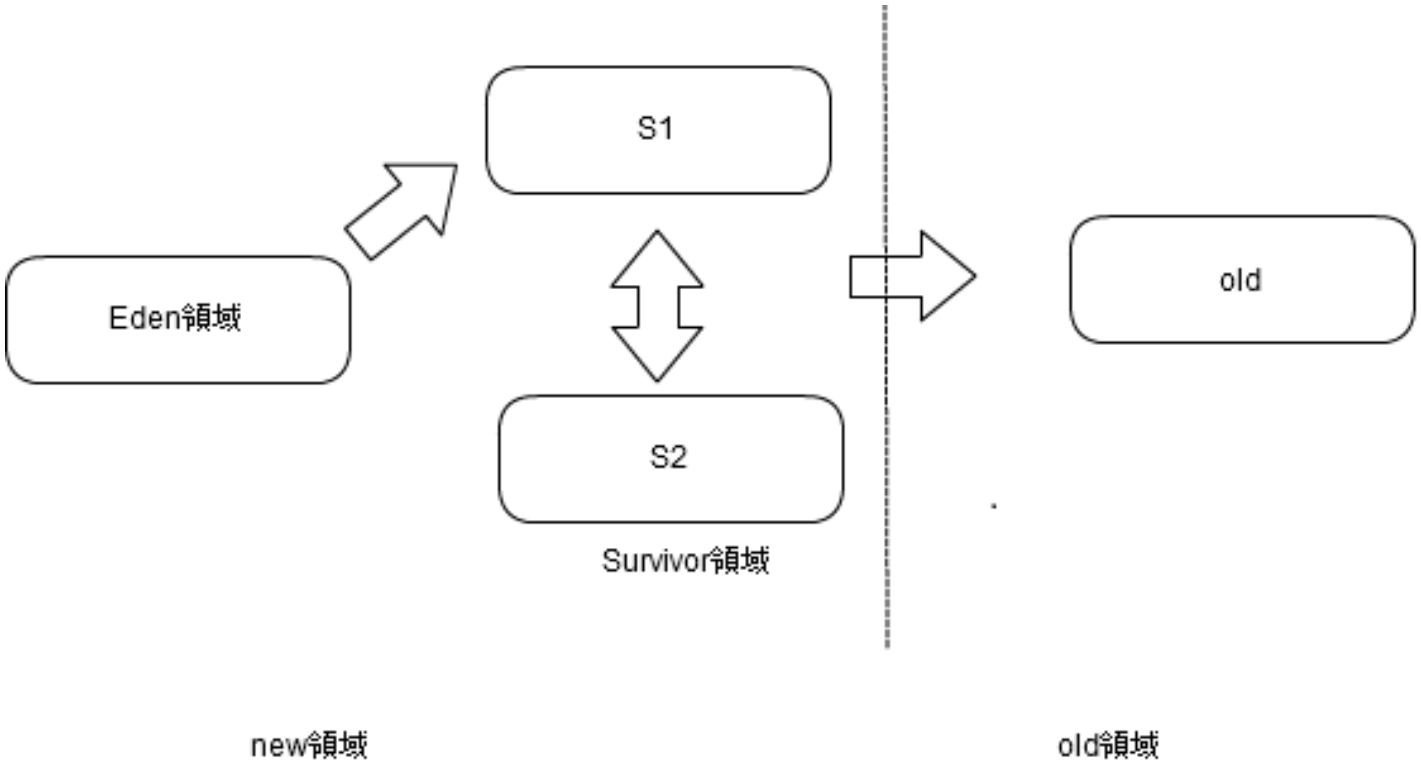
- 例：Tomcatの起動時間
  Tomcat7.0.47 JDK7u45(64bit)
  - 指定なし
    → 1510ms
  - -XX:+TieredCompilation -XX:TieredStopAtLevel=1
    →602ms
- 更に、バイトコードの検証をスキップすると…
  - -XX:+TieredCompilation -XX:TieredStopAtLevel=1 -Xverify:none
    →556ms

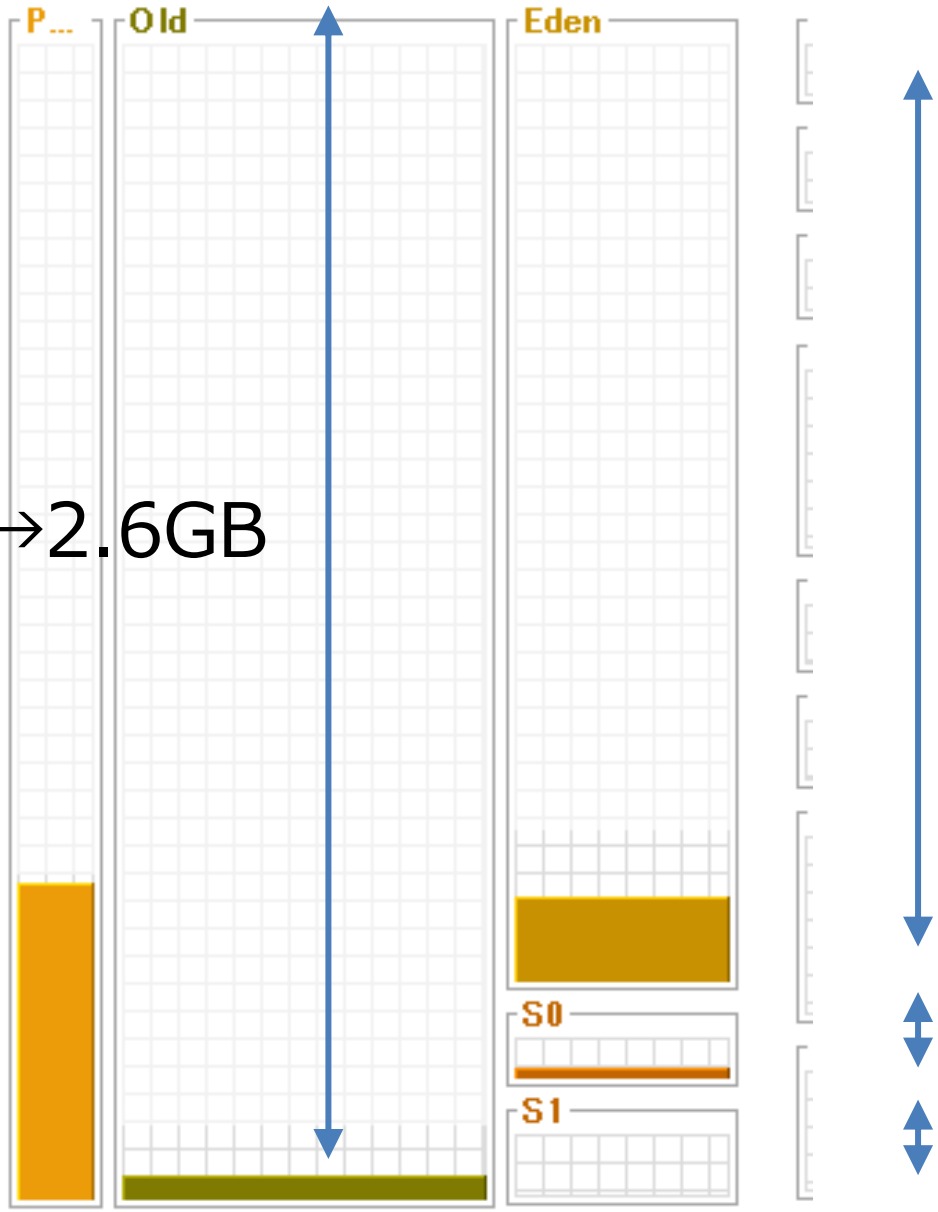
サーバーVMとクライアントVMの違い

- ヒープ容量を指定しなかった場合のデフォルト値
  - クライアントVM→64MB
  - サーバーVM→ 物理メモリーの 1/4 か、1GB かの小さい方。
    - また、パフォーマンス関係のパラメーターは、統計情報に応じて、動的に変化します。(エルゴノミクス)

- じゃあ、エルゴノミクス任せにすればいいのかというと…

(おさらい)世代別GC





Old→2.6GB

Eden→1.3GB

S0→450MB

S1→450MB

Let’sチューニング

- ヒープのパラメーターを調整
- ただし、細かい値の設定はエルゴノミクスに任せる

- -Xms ...ヒープの初期容量
- -Xmx ...ヒープの最大容量
- -XX:NewSize ... new領域のサイズ
- -XX:NewRatio new領域とold領域の比率
- -XX:SurvivorRatio ...EdenとSurvivorの比率
- -XX:targetSurvivorRatio ... Survivorが一杯になったと判定される閾値
- -XX:maxTenuringThreshold ... new領域からold領域に移動する閾値

- フルGCの発生をどれだけ回避するかが鍵。
  - 但し、ライフサイクルの長いオブジェクトの存在や、定期フルGCのこともあるので、フルGCを起こさないチューニングというのは非現実的

- CPUのリソースを潤沢に使えるなら、コンカレントGCの使用も手でしょう。
- 但し、従前のGCとコンカレントGCではGC関連のデフォルトパラメーターが異なるので、そこには注意。

さらに注意しなければならないのは、コンカレントGCオプション「-XX:+UseConcMarkSweepGC」を使用した場合、New世代領域の オプションデフォルト値が「-XX:SurvivorRatio=1024 -XX:MaxTenuringThreshold=0」に自動設定されるという点です。
http://wall-climb.com/2009/10/12/%E3%82%B3%E3%83%B3%E3%82%AB%E3%83%AC%E3%83%B3%E3%83%88gc%E3%81%AE%E6%B3%A8%E6%84%8F%E7%82%B9/

→この記述自体は obsoleted

- jdk7u45の場合、 MaxTenuringThreshold がコンカンレントGCの場合は8(通常は15)

(小ネタ)mod_proxy_ajpの落とし穴

(小ネタ)ヒープリークの調査方法

- ストレスツールを使って、一定の負荷をかけながら、定期的にヒープの統計情報を取得する
- http://qiita.com/setoazusa/items/ee0a7d795c2687b80817